操作系统 · 请求分页与软硬件互斥 · 考点精讲

408 各科大题知识点强化课 | 课程第 12 讲（分 P14） | 时长 3 小时 12 分 25 秒

本讲范围（前半场 · 第三章）：请求分页——虚拟存储的「错觉」、局部性原理、**页表项比基本分页多出的四个字段**、**缺页中断属于内中断**、驻留集与工作集、三种内存分配策略、**页面置换算法 OPT / FIFO / LRU / CLOCK / 改进CLOCK 的完整执行过程**（含 Belady 异常、改进CLOCK 的四步与淘汰顺序），以及讲义第 15、19、51、55 题。

本讲范围（后半场 · 第二章补漏）：上一讲（PV 操作）末尾预告要补的「**实现临界区互斥的软硬件方法**」——临界区四部分与四条准则、**单标志法 / 双标志先检查 / 双标志后检查 / Peterson** 四种软件方法（含代码与「谁能互斥、谁不满足哪条准则」的逐条判定）、**中断屏蔽 / TS 指令 / Swap 指令**三种硬件方法，以及讲义第 25、52 题。

读法建议：置换算法**必须自己把表画一遍**——原话「只有说亲自去画那个流程，就什么都清楚了」；改进 CLOCK 他要求「刻意地多去重复个三四遍」。互斥部分**对着一行行代码看**，不要只看文字。

一句话：最值钱的两条是「**改进CLOCK 淘汰顺序 (0,0) → (1,1) → (1,0) → (1,1)**」与「**只有记录型信号量的 P 操作能实现让权等待，其余全是忙等**」。

目录

1 考情分析：本讲定位、时间轴与复习优先级

2 核心概念：请求分页的机制与临界区的准则

3 解题方法论：置换算法全推演 + 互斥方法判定

4 典型题型与模板：讲义第 15 / 19 / 51 / 52 题

5 统考真题精解：2021 年第 28 题与第 45 题（附年份认定依据）

6 易错点清单

7 考场速查卡

附 本资料的来源与边界

ASR 校正说明（读引用前先看这一段）

转写稿由本地 Whisper 生成，本讲**错字密度极高且集中在术语上**。引用时统一校正，主要替换：

分页类：请留分业 / 请问有分页 / 运行分页 → **请求分页**；夜框 / 叶框 / 液宽 / 页旷 / 叶方 / 页方 → **页框**；液面 / 夜面 / 叶面 → **页面**；缺业 / 缺益 / 缺盐 / 缺眼 → **缺页**；支换 / 置话 / 之换 / 字换 / 自换 / 制划 / 智患 → **置换**；物理矿 / 物理框 → **物理块**；逐流级 / 注流级 / 铸流级 → **驻留集**；工作级 → **工作集**；夜幕录 → **页目录**；快标 → **快表**；贝拉迪 → **Belady**。

最有欺骗性的一处：`clog` / `club` / `claw` / `Cloud` → **CLOCK**；而「**时冬之换**」（`[00:32:24]`）与「**十中置换**」（`[01:10:58]`、`[01:11:15]`）都是「**时钟置换**」的同音错写，仍指 **CLOCK**，**不是 LRU**（判断依据见文末不确定清单⑨）；「分十人」（`[01:11:15]`）按上下文是「顺时针」。

互斥类：邻届 / 连接 / 零件 / 灵界 / 凛介 / 林界 → **临界**；忽斥 / 护车 / 护士 / 互职 / 后置 / 护持 → **互斥**；盲者等待 → **忙则等待**；让全等待 / 让钱等待 → **让权等待**；空前让进 → **空闲让进**；机会现象 / 机构现象 / 计划现象 → **饥饿现象**；冲断屏蔽 → **中断屏蔽**；太史兰的赛 / 太史材的赛法 / DS 指令 / 剃暗的 → **TestAndSet（TS/TSL）指令**；swime / swift / 护手指令 → **Swap 指令**；碳 / 吞 / TEN / 凳 / 特别量 → `turn`；局务变量 / 折位错 → 局部变量。

ASR 校正说明（续）：布尔值这一条最容易读反

true 被写成「**错 / 处 / 粗 / 触**」——如「把 `flag1` 改为错」= `flag[1] = true`、「`flag1` 等于处这个循环就会一直循环」= `while(flag[1])` 成立；false 被写成「**force / foss / False**」。

⚠️ 但 `[02:16:46]` 有一句「那一开始 `flag0` 是错，是 force 没问题」——同一句里「错」与「force」并列自相矛盾，而他在 `[02:15:56]` 明确说过「`FLAG0` 和 `FLAG1` 都是 false」。**凡遇冲突，一律按代码逻辑（双标志法两个标志初值都是 false）处理**，正文不采信这一句。

1 考情分析：本讲定位、时间轴与复习优先级

1.1 这一讲在课程里的位置

本讲是《408 各科大题知识点强化课》的第 14 个分 P、课程内第 12 讲，时长 **3 小时 12 分 25 秒**，是**跨两章拼起来的一讲**：前半场继续第三章（存储管理），后半场回来补第二章的尾巴。

> 「我们今天就主要还是讲第三章，然后以及 PV 操作。上一次课主要讲了这个分页，一级页表二级页表这些，今天看一下请求分页。」（`[00:08:35]`–`[00:09:06]`）
> 「接着往后看一下这个同步和互斥。**之前落的一些东西没讲**……上次上课只讲了 PV 操作，今天把一些当时没提到的再讲一下。」（`[02:03:53]`–`[02:04:09]`）

时间段	内容	性质
`00:00:04`–`00:08:10`	考研报名提醒、408 择校资料与院校地图、分数线公众号	非考点
`00:08:12`–`00:31:29`	请求分页的引入、虚拟存储、局部性、页表项、缺页中断、驻留集/工作集、三种分配策略	概念题 + 大题铺垫
`00:31:35`–`01:29:17`	**页面置换算法：OPT、FIFO（Belady）、LRU、CLOCK、改进CLOCK**	**本讲最重，大题与选择题都有**
`01:29:30`–`01:49:30`	讲义第 51、55、19 题；跨科综合图（地址变换 + Cache）	大题模板
`01:50:06`–`02:03:47`	**课间休息**（约 13 分 40 秒，无内容）	—
`02:03:53`–`02:28:16`	临界区四部分与四准则、四种软件互斥方法、三种硬件互斥方法、让权等待	选择题高频 + 大题
`02:28:28`–`03:07:00`	讲义第 52 题（Peterson）、第 25 题（整型信号量 + 开关中断）	大题模板
`03:07:03`–`03:12:03`	复习节奏建议（10 月 20 号开始做整套卷）、读者写者「大后期再看」	非考点

1.2 老师给的分量排序（按这个顺序投入时间）

> 「这些东西相对来说没有之前讲的重要……但是学到后期的话你可以作为……**我们复习全面的话也要把这些东西看一看。他们属于也考过但是频率要低一点**。如果前面的都学好了，应该把这个也看一看；如果前面的学得还不太好，就可以先去**以前面的为主**。」（`[02:28:39]`–`[02:29:04]`，讲互斥的软硬件方法）

内容	分量	他让你做的事
页表项、缺页中断性质	**必记**	「这四个字段要记下」「缺页是属于内中断，这个也是在整体的真题里体会过很多次」
OPT / FIFO / LRU	不难，但要动手	「前面几个我觉得都不是难点」，但「**课下一定要把这几个表格都动手写一写**」
CLOCK / 改进 CLOCK	**本讲最难、细节最多**	「前三个……学过一遍两遍就记得特别牢，但是 CLOCK 的细节比较多……**一定要刻意地多去重复个三四遍**」「课下一定要再写写画画，不要只看文字」
三种分配策略名词	低频但出现过	「这几个分配策略是出现过的，选择题出现过，大题也是出现过这个说法的」
互斥的四种软件 + 三种硬件方法	低于 PV	「**一定要结合代码去理解这些特点**」「这种代码不需要你把它背（下来），但是你得多看几遍」
让权等待	反复强调 4 次	「**能实现让权等待的只有信号量方法**」（`[02:33:55]`、`[03:05:02]`）

2 核心概念：请求分页的机制与临界区的准则

2.1 从连续分配到请求分页：虚拟存储是怎么「骗」出来的

> 「连续分配就是一个进程，你把它放到内存，**整个进程是连续的**；非连续分配就是……这个进程有多个页面，**不同的页面之间是可以不连续的**。」
> 「基本分页就是虽然它不连续，但是这个进程**三个页面全占内存**；请求分页就是……**只把一个进程的一部分放到内存**。」（`[00:10:32]`–`[00:11:52]`）

他举的「错觉」例子（`[00:12:11]`–`[00:15:25]`）：内存只有 4 个页框，P1、P2 各 4 个页面，各只装入一部分（P1 装 0、1；P2 装 2、3），跑起来「你就感觉 P1 和 P2 是同时执行的」「你就会觉得内存是 8 个」——其实只有 4 个。

老师给虚拟存储的一句话定义

「**虚拟存储就是说实际（内存）容量未变，但是你用的时候你就感觉变大了**，因为你每个进程只调了一部分数据进去，但你会以为每个进程是把全部调进去了。」（[00:15:25]）

两个动作：访问的信息不在内存 → 操作系统从外存调入，叫**请求调页**；内存空间不够 → 把不用的换出去，叫**页面置换**。他强调「一般这个置换是在**自己进程的空间内**去置换」（[00:17:03]–[00:18:12]）。

局部性原理（一切置换算法的依据）：「时间局部性是因为程序中有大量的循环；空间局部性是因为指令是顺序存放的，包括数据一般是以数组的形式。」（[00:16:28]–[00:16:46]）

2.2 请求分页的页表项：比基本分页多出的四个字段（第一个必背点）

「之前你学的是基本分页，**页表只需要有页号和物理块号**就行了。然后现在请求分页的话，**首先你得有一个状态位**，说明是否在内存；**要有一个访问字段**，记录一下最近的访问次数，它是供后面用置换算法的时候再用；**还有一个叫做修改位**……（置换时）如果发生更改还要把它修改回外存；**最后一个就是如果缺页的话你肯定要从外存里面把它调进来，你还得记录一下这个在外存上的哪些位置**。这四个字段要记下，比基本分页它多了这四个东西，得知道每一个的含义。」（[00:20:00]–[00:21:26]；括号内按上下文补）

```
请求分页页表项 = 基本分页（页号 + 物理块号）+ 以下四个 ——
  ① 状态位（存在位 / 有效位）：该页是否已在内存     → 决定要不要发缺页中断
  ② 访问字段（访问位）        ：记录最近被访问的情况 → CLOCK / LRU 靠它工作
  ③ 修改位（脏位）            ：装入后是否被改过     → 决定换出时要不要写回外存
  ④ 外存地址                  ：该页在外存的位置     → 缺页时知道去哪儿调
记忆锚点：在不在（状态位）| 最近用没用（访问字段）| 改没改（修改位）| 在外存哪（外存地址）
```

2.3 缺页中断：属于内中断，发生在指令执行期间

「**缺页是属于内部中断**，这个也是在整体的真题里体会过很多次。」

「**这个缺页中断发生在指令执行期间，不是执行后**，主要是内部异常。**内中断是在指令执行过程中产生的**；外中断是一条指令执行之后才能检测到的。」

「**一条指令是可以执行多次缺页中断**。回忆一下组成原理里学过的二地址指令、三地址指令，如果说你操作码里面是有多个地址码的，比如三个地址码，那你每个地址都有可能缺页，所以一条指令是可能多次缺页中断。」（[00:21:36]–[00:22:52]）

⚠ 三个最常被写错的点

① 缺页中断 = **内中断（内部异常）**，不是外中断；

② 它产生在**指令执行期间**，属「故障 / 异常」这一类，不是「指令执行后检测到」的外中断；

③ 一条指令**可能触发多次缺页中断**（多地址码指令每个地址都可能缺页），不要写「一条指令最多一次缺页」。

驻留集与工作集（必须成对记）：「给一个进程分配的**页框的集合叫驻留集**……**这个驻留集和后面的工作集两个概念一定要放一块去学习对比。**这个**工作集是（经常）访问了页面的集合**」「**一般是驻留集不能小于工作集，否则会频繁缺页**」（[00:22:59]–[00:25:15]）。⚠ 他把工作集说成「经常用的页面的集合」，教材的

标准定义是「某段时间间隔内进程实际访问的页面的集合」（**窗口概念**）；答题若考定义以教材为准（见文末「教材补充」）。

2.4 三种内存分配策略（选择题与大题都出现过「这个说法」）

策略	老师的一句话核心	区别点
固定分配局部置换	「每个进程分配的页框是固定的。如果发生缺页，它就只能**从已经分配的（页框）给它选一个置换出去**」	一开始分几个，后面**永远不变**
可变分配全局置换	「先给进程分配一定的数量，然后如果发生缺页，就从**空闲物理块堆列里取出给它**，就说动态地增加物理块」	**只要缺页就多给一个**，不动已有页框
可变分配局部置换	「如果发生缺页**从已经分配的里面置换**；**如果频繁地缺页中断，那就再给它多分一个**……反之如果是缺页率特别低，再给它减少」	前两种的**结合**：先局部置换，**频繁**缺页才加，缺页率低还会减

2.5 后半场起点：临界资源、临界区的四个部分、四条准则

> 「**临界资源就是说一次只让一个进程用的（资源）**，然后临界资源的访问必须互斥。访问临界资源的代码叫临界区……**分成了这样的四个部分：进入区、临界区、退出区、剩余区。**」（`[02:04:10]`–`[02:04:37]`）

```
四条准则（后面每一种方法都要回答「它不满足哪一条」）——
 空闲让进：「如果临界区空闲……这个进程就能进到临界区」（如打印机空闲时谁都能用）
 忙则等待：「如果别的进程正在临界区，他就得等」                    （转写作「盲者等待」）
 有限等待：「他不能无限等，但是（在）有限的时间（内能等到）」
 让权等待：「如果自己不能进到临界区，他就会把 CPU 让出来，本来是执行的然后变成阻塞态」
```

3 解题方法论：置换算法全推演 + 互斥方法判定

3.1 共同的题设与表格读法

本讲 OPT / FIFO / LRU / CLOCK 用的是**同一个页面走向**（他**没有逐字念出整串**，但按他推演的每一步可唯一还原；见文末不确定清单①）：

```
页面走向（共 20 次访问）：
  7  0  1  2  0  3  0  4  2  3  0  3  2  1  2  0  1  7  0  1
题设：给该进程分配 3 个物理块（页框）；CLOCK 那一组给 4 个物理块。
表中「√」= 缺页，「·」= 命中（老师原话：「打的这个对号就是缺页了」）。
FIFO 表按先进先出队列顺序排列，【最左端就是下一个被淘汰的】；OPT/LRU 表的排列顺序不代表淘汰顺序。
```

> 他要求：「**课下把这个表格自己写一遍，上面保留、下面再写物理块一、物理块二、物理块三和缺页**……有时候选择题就会让你画这种表格」（`[00:34:51]`、`[00:38:47]`）。

3.2 OPT 最佳置换：往后看，淘汰「最长时间内不再被访问」的

页面走向	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
块1	7	7	7	0	0	0	0	2	2	2	2	2	2	2	2	2	2	0	0	0
块2		0	0	1	1	2	2	3	3	3	3	3	3	0	0	0	0	1	1	1
块3			1	2	2	3	3	4	4	4	0	0	0	1	1	1	1	7	7	7
缺页	√	√	√	√	·	√	·	√	·	·	√	·	·	√	·	·	·	√	·	·

> 「最开始你访问 7 的时候是缺页，因为一开始给他分的三个框都是空闲的，先把 7 放进去；接下来 0 的时候又缺页，再把 0 放进去；第三是 1 的时候继续把 1 放进去，1 也是缺页了……**如果现在你要访问 2，这个进程分的三个页框已经用完了，放的 7、0、1，你要用 2，2 没地方放就得置换一个。**」
> 「最佳置换就是往后看，从当前的时间往后看。**比方说后面你 0 还要用到，肯定不要置换 0；但往后看的话这个 1 也还要用到，那也不要置换 1；但是 7 就最长一段时间都不用，那你就先把 7 置换。**」
> 「然后是 0 的时候——前面放的 2、0、1 有 0，那叫命中了……接下来到 3，3 不在 2、0、1 里面，那说明缺页了，那你往后看，这个 0 和 2 后面还要用到不能置换，**1 是最长一段时间都不会用的，就把 1 置换了。**」
> （`[00:33:35]`–`[00:34:39]`）

> ⚠ **他专门澄清：「缺页」与「置换」不是一回事**
>
> 「发生缺页的是页面置换的那个（时刻），**前三次缺页没有置换**，后面的每一次都要置换了。因为一旦到后面都放满了，你肯定缺页就得置换；前三次它属于缺页了，但是还有空闲空间……**属于缺页次数和置换次数的两个概念。**」（`[00:35:05]`–`[00:35:25]`）
> 本表：全 20 次共缺页 **9 次**（前 3 次缺页不需要置换）。⚠ 老师只推了前 6 步并说「后面的流程都一模一样」，**完整 20 列的表与缺页次数是整理时按他给的规则补全的**（与教材标准结果一致）。

3.3 FIFO 先进先出：谁先进来谁先出去（＋Belady 异常）

页面走向	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
块1（最旧）	7	7	7	0	0	1	2	3	0	4	2	2	2	3	0	0	0	1	2	7
块2		0	0	1	1	2	3	0	4	2	3	3	3	0	1	1	1	2	7	0
块3（最新）			1	2	2	3	0	4	2	3	0	0	0	1	2	2	2	7	0	1
缺页	√	√	√	√	·	√	√	√	√	√	√	·	·	√	√	·	·	√	√	√

> 「先进先出也是比较简单的，最开始 7 直接放进去，然后因为一开始三个都是空闲的，0 缺的时候直接放进去，1 的时候缺的时候直接放进去。到 2 的时候要置换了，**先进先出，谁先进来谁先出去**，7 是最先进来的，那 2 就把 7 置换，变成 2、0、1，缺页了。现在访问 0 的话里面就有 0，那就没有缺页，所以这一步就没有缺页，里面依然是 2、0、1。」
> 「到 3 的时候又缺页了，里面是 2、0、1……（**改口**）那这个 0 是最先进来的……**如果这里发生缺页的话，3 应该把这个 0 置换，因为 0 是最早进来的**……下一个阶段当前放的是 2、3、1（原文如此，按表应为 1、2、3），你现

> 在访问 0，又缺页了，这个 1 很早就进来了，然后再进来 2 再进来 3，**所以这个 0 会把 1 置换。**」（`[00:36:49]`–`[00:38:35]`）

> ⚠ **老师在这里自己改口了一次，改口才对**
>
> 他先说「这个 **1** 是最先进来的，所以 3 会把 **1** 置换」，随即改口「那这个 **0** 是最先进来的……3 应该把 **0** 置换」。
> **为什么 0 对：**访问 2 时淘汰的是 7，队列变成 0、1、2；接着访问 0 是**命中**——**FIFO 命中时不改变队列顺序**，所以队列仍是 0、1、2，最旧的还是 0。改口后的答案与教材一致。

> **Belady 异常：只发生在 FIFO**
>
> 「先进先出有一个东西叫**贝拉迪异常**，这个就记一下就行——**就是给他分配的物理块变多，反而缺页次数不减反增。**」
> 「本来（给）三个物理块的时候，它缺页也缺了一二三四五六七八九，缺了九次；现在都改到四个物理块了，反而变了十次。**物理块多了，你缺的次数也变多了，这叫贝拉迪异常。**」（`[00:38:59]`–`[00:39:52]`）

下面是教材上与 9 / 10 这两个数字相符的经典例子（⚠ **他只念了结论，没有念这组例子的页面走向**，序列是按标准例子还原的，见文末不确定清单③）：

页面走向	3	2	1	0	3	2	4	3	2	1	0	4
3 块 · 块1	3	3	3	2	1	0	3	3	3	2	4	4
3 块 · 块2		2	2	1	0	3	2	2	2	4	1	1
3 块 · 块3			1	0	3	2	4	4	4	1	0	0
3 块 · 缺页	√	√	√	√	√	√	√	·	·	√	√	·
4 块 · 块1	3	3	3	3	3	3	2	1	0	4	3	2
4 块 · 块2		2	2	2	2	2	1	0	4	3	2	1
4 块 · 块3			1	1	1	1	0	4	3	2	1	0
4 块 · 块4				0	0	0	4	3	2	1	0	4
4 块 · 缺页	√	√	√	√	·	·	√	√	√	√	√	√

3 块共 9 次缺页、4 块共 10 次——物理块变多，缺页反而多了 1 次，这就是 Belady 异常。本讲 FIFO 那 20 次访问共缺页 **15 次**（整理时补全）。

3.4 LRU 最近最久未使用：往前看，淘汰「最久没用过」的

> 「这个 LRU 出现的频率是最高的。有的地方会叫最近最长时间未使用，其实你也可以直接把它叫做**最久未使用。**」
> 「LRU 就是因为局部性原理——**你最近用过 1，后面可能还要用 1**……**在这个 LRU 是往前看的，从当前的位置往前**

> 看：1 最近刚用过，0 也最近刚用过，7 很长时间没用，所以叫最久未使用，2 会把 7 置换。」（`[00:40:01]`–`[00:41:37]`）

页面走向	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
最久未用	7	7	7	0	1	2	2	3	0	4	2	2	0	3	3	1	2	0	1	7
次久		0	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0
最近刚用			1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
缺页	√	√	√	√	·	√	·	√	√	√	√	·	·	√	·	√	·	√	·	·

全 20 次共缺页 **12 次**。⚠ 老师同样只推了前 6 步（7、0、1 装入 → 2 换出 7 → 0 命中 → 3 换出 1）并说「后面的流程都一模一样」，**完整表与次数为整理时补全**。

3.5 CLOCK（简单时钟）：给每个页面一个访问位，转圈找 0

> **老师的机制描述（原文）**
>
> 「首先这个 CLOCK 算法要先画一些东西：**首先每个页面有一个访问位，然后首次被装入或者被访问（时）置为一**；然后内部的话有一个类似于**循环队列**；然后再往后，当某一页被替换，**指针会指向被替换的下一个**；如果需要淘汰，只需要检查访问位，**如果为零就换出，如果为一就把它置为零**。」（`[00:45:51]`–`[00:46:27]`）
>
> 「**指针的初始指向位置和方向（顺时针 / 逆时针）题干一定会告诉你，不然就没法做**。」（`[00:51:18]`）
>
> 「**只有（发生）淘汰，指针才会动**；被访问（命中）只会把访问位从零改到一，指针不动。」（`[00:54:31]`）
>
> 「**这个表格和前面不一样了**——前面每一个访问序列它只有一列，现在它是有两列，后面这个东西是访问位……**这个表格课下也要能自己写出来**。」（`[00:46:39]`）

他的例子是 **4 个物理块**，指针初始指向第一个页框，页面走向同 §3.1。下表是他**逐次推演的前 15 次**（后 5 次他没讲）：

次序	访问	判定	指针扫描过程	四个页框（页面(访问位)）
1	7	缺页	有空框，直接装入，访问位置 1	7(1) — — —
2	0	缺页	有空框，直接装入	7(1) 0(1) — —
3	1	缺页	有空框，直接装入	7(1) 0(1) 1(1) —
4	2	缺页	有空框，直接装入	7(1) 0(1) 1(1) 2(1)

次序	访问	判定	指针扫描过程	四个页框（页面(访问位)）
5	0	命中	指针不动，只把 0 的访问位置 1（本来就是 1）	7(1) 0(1) 1(1) 2(1)
6	3	缺页	指针从 7 开始：7、0、1、2 的访问位全是 1 → 逐个清 0 → **转一圈回到 7**，其访问位已是 0 → **淘汰 7，装入 3**；指针指向下一个	3(1) 0(0) 1(0) 2(0)
7	0	命中	指针不动，0 的访问位 0 → 1	3(1) 0(1) 1(0) 2(0)
8	4	缺页	指针从 0 开始：0 的访问位=1 → 清 0 后移；**1 的访问位=0 → 淘汰 1，装入 4**	3(1) 0(0) 4(1) 2(0)

9–132 3 0 3 2全命中五次都命中：访问谁就把谁的访问位改为 1（本来就是 1 的不变），**指针不动**3(1) 0(1) 4(1) 2(1) 141缺页指针从 2 开始：2、3、0、4 的访问位全是 1 → 逐个清 0 → **转一圈回到 2**，其访问位已是 0 → **淘汰 2，装入 1**3(0) 0(0) 4(0) 1(1) 152缺页指针从 3 开始：3 的访问位=1 → 清 0 后移；**0 的访问位=0 → 淘汰 0，装入 2**3(0) 2(1) 4(0) 1(1)

> **「就是关于 CLOCK，你要学到这个地步，它其实考试无论怎么考都没问题。」**（`[00:58:12]`）

前 15 次共 8 次缺页（次序 1、2、3、4、6、8、14、15）。⚠️ 老师没有给总次数，这是整理时按他讲到的范围加的计数。

3.6 改进CLOCK：多一个修改位，四步流程 + 淘汰顺序（本讲最该背的一条）

> 「这个改进的 CLOCK 算法和之前其实很接近，只不过它是多了一个——**除了访问位额外增加了一个叫修改位。**」「首先这个步骤是第一步：**他去找既没访问又没修改（A=0 且 M=0）的把它置换，第一次扫描不会改变**（标志位）；**第二次扫描的话会把所有的访问位置为 0**，这个和经典的 CLOCK 是一样的；**如果还没有找到那就重复**——如果还没有找到就重复第一步，如果有必要重复第二步。」（`[01:19:41]`–`[01:20:47]`，A = 访问位，M = 修改位）

四步流程（他要求「这里的流程要记下」）+ 结论

> ① 第一轮：从指针当前位置扫描，找 (A=0, M=0) 的页面 → 找到就淘汰；本轮【不修改】任何标志位。
> ② 第二轮：若①失败，扫描找 (A=0, M=1) 的页面，并把扫描过的页面 A 置 0。
> ③ 第三轮：若②仍失败，重复第①轮（此时所有页面 A 都已是 0，一定能找到）。
> ④ 第四轮：若③仍失败，重复第②轮。
> 每次需要淘汰，都要【从头】把这四步走一遍，不要接着上次走。
>
> 淘汰顺序（结论）：(0,0) → (0,1) → (1,0) → (1,1)
> 即：又没访问过又没改过的最先被淘汰，又访问过又改过的最后被淘汰。
> 新调入的页面：访问位 = 1、修改位 = 0；中途被修改过才把修改位置 1。

> ⚠ **这句话转写里有他的一次「说反了又改口」**
>
> `[01:24:34]` 转写是「其次是访问了但没修改」，`[01:24:36]` 紧接一句「但是修改了就没有访问但是修改了」——即他**当场把第二、第三位对调了回来**：第二位是「没访问但修改了 (0,1)」，第三位才是「访问了但没修改 (1,0)」。
> **本资料取 (0,0) → (0,1) → (1,0) → (1,1)**，依据三：① 他改口后的说法；② 他随后用四个状态推导出来的正是这个次序（「00 最先出去」「一开始 01 已经换出去了」「接下来是 10 就出去了」「最后这个位置（11）他是最后出去的」，`[01:25:41]`–`[01:28:57]`）；③ 与教材结论一致。⚠ 他念顺序时的那串数字在转写里是乱码（「0010011」「001 1001」），无法逐字复核。

> **他做的极端例子：四个页面的 (A,M) 全是 (1,1)**
>
> 「那第一步就是找 00，这第一步你转了一圈（都没找到）；第二圈的话……第二步就是把访问位置为 0 再往后走，第二步是找 01，还没找到，把它改成 0；这时候第二圈结束，再来看第三圈，第三圈继续重复第一步，找 00，所以第三圈的时候又会发现转了一圈还是没找到；然后到了第四圈就可以了，第四圈就重复第二步，第二步就找 01，那现在就找到 01 了。」（`[01:21:37]`–`[01:23:18]`）

3.7 五种置换算法横向对比 + 学习重心

算法	淘汰谁（一句话）	看的方向	需要什么硬件位	Belady 异常
OPT	未来最长时间不再被访问的	往后看	需要预知未来（不可实现，只作理论下界）	无
FIFO	最先进入内存的	不看（用队列）	一个队列	**有（唯一）**
LRU	最久没被访问的	往前看	需记录访问历史（栈 / 计数器），开销大	无
CLOCK	转圈找访问位为 0 的	用访问位近似「最近」	访问位 + 循环队列 + 指针	无
改进 CLOCK	按 (0,0) → (0,1) → (1,0) → (1,1)	访问位 + 修改位	访问位 + 修改位 + 指针	无

「前三个学完一遍一般不太会忘……**但是 CLOCK 的细节比较多，很迷惘，一定要多学几遍**。像刚才前面三个你可能学一遍或者两遍就行，**这个的话一定要刻意地多去重复个三四遍**。」（`[00:42:33]`–`[00:42:57]`）

3.8 地址变换三步（第三章真正的大题核心）

> 「其实整个第三章核心要讲的一个东西就是地址变换——**把逻辑地址或者虚拟地址变成物理地址**。」
> 「让你求物理地址……**无非就是把页号替换成页框号**……它背后的本质：**你用页框号乘以了一个（页面）大小，得到这个页框的首地址，再加上了一个偏移量**。」
> 「逻辑地址转物理地址，这个偏移量是不变的……**这个偏移量（页内）相对位置其实没有变**。」（`[01:03:55]`–`[01:10:24]`）

三步：
① 拆逻辑地址：高位 = 页号 P，低位 = 页内偏移 W（位数由页面大小决定：4KB → 12 位，1KB → 10 位）
② 查页表（或快表）得到页框号 F —— 缺页就先按置换算法换入，再拿新页框号
③ 拼物理地址：物理地址 = F × 页面大小 + W （等价于把 P 换成 F，W 原样保留）
页号位数 = 逻辑地址位数 − 偏移位数。

3.9 时间开销：缺页率与 EAT（⚠ 本讲没有讲，属教材补充）

⚠ **重要边界：这两个公式在本讲转写稿里一次都没有出现**

「EAT 有效访问时间」「缺页率」的公式，**在 3 小时 12 分的讲稿里全文检索不到**（无「有效访问」「EAT」的任何推演，也没有缺页率公式）。他只在一处顺口提到过「缺页率」这个词——讲「可变分配局部置换」时说「**反之如果是缺页率特别低，再给它减少**」（`[00:31:07]`，转写作「推业率」）。下面这组公式**是教材补充，不是老师口径。**

【教材补充，非本讲内容】
缺页率 p = 缺页次数 / 总访问次数；　　命中率 = 1 − p
EAT（有效访问时间）= 命中时的访存时间 + 缺页率 × 缺页处理开销
常见写法：EAT = (1 − p) × 访问一次内存的时间 + p × 缺页处理时间
其中　缺页处理时间 ≈ 缺页中断处理时间 + 再从内存访问一次的时间
若系统还有快表（TLB），按「快表命中 / 快表未命中但页在内存 / 缺页」三段相乘相加。

3.10 互斥方法判定套路（后半场的一张总表）

判定四步：
① 先看「检查」与「置标志」的先后：先检查 = 双标志先检查；先置标志 = 双标志后检查。
② 再找反例：能否构造出「两个都进临界区」（→ 不满足忙则等待）或「两个都进不去」（→ 不满足空闲让进、有限等待）。
③ 最后问：进不去时进程是【卡在 while 里空转】还是【阻塞并让出 CPU】？空转 = 不满足让权等待。
④ 结论：本讲全部方法里，只有记录型信号量的 P 操作满足让权等待。

方法	能否互斥	空闲让进	忙则等待	有限等待	让权等待
单标志法	**能**	✗ 必须交替使用	✓	✓	✗
双标志先检查	✗ 可能同时进	✓	✗	✓	✗
双标志后检查	**能**	✗	✓	✗ 会饥饿	✗
Peterson	**能**	✓	✓	✓	✗
中断屏蔽（关中断）	**能**（单 CPU）	✓	✓	✓	✗
TS 指令 / Swap 指令	**能**	✓	✓	✓	✗ 忙等
整型信号量	**能**	✓	✓	✓	✗ 忙等
记录型信号量 P 操作	**能**	✓	✓	✓	✓（唯一）

⚠ 上表按老师逐条口述整理，✓/✗ 只覆盖他明确点过的准则；「双标志先检查不能互斥」与教材一致。

4 典型题型与模板：讲义第 15 / 19 / 51 / 52 题

4.1 第 15 题：固定分配局部置换 + FIFO / CLOCK 求物理地址

题面（按他念出的部分整理）：逻辑地址空间与物理地址空间均为 64KB，按字节编址；进程最多需要 6 个页面，每页 1KB；采用**固定分配局部置换**，占有 **4 个页框**。装入与访问：时刻 130 把**页 0** 装入**页框 7**；160 把**页 3** 装入**页框 9**；200 把**页 2** 装入**页框 2**；230 把**页 1** 装入**页框 4**。问时刻 260 访问某逻辑地址时：(1) 页号是多少；(2) 分别用 FIFO 和 CLOCK 求物理地址。

```
先拆地址：64KB 按字节编址 → 逻辑/物理地址各 16 位；每页 1KB → 偏移 10 位、页号 6 位
所以：物理地址 = 页框号(6 位) << 10  |  低 10 位偏移（页号换成页框号，偏移不变）

(1) 页号 = 5                ← 把逻辑地址的二进制写出来，高 6 位就是页号
(2) FIFO：最先进去的是【页 0】（时刻 130 最早）→ 页 5 顶掉页 0 → 页 5 使用【页框 7】
    CLOCK：4 个页框的访问位初始都是 1（刚装入）→ 逐个把 1 改成 0、指针后移，
           转完一圈回到起点，其访问位已为 0 → 页 5 把【页 2】换出 → 页 5 使用【页框 2】
```

> **他专门提醒的答题规范（这条比答案值钱）**
>
> 「但这个题目明确说了要求给出计算过程，所以**考试千万不要（只）写个答案**，就说把这个页号替换成页框号。你要分析一下**为什么这个页框号是 7**，要通过上面去分析先进先出算法——**因为 0 号是最先进来的，放到了 7 号，所以置换的话应该先把 0 号置换出去**，要稍微写一下过程。」（`[01:10:37]`–`[01:10:56]`）
>
> **他还补了第三问可能怎么问：**「如果这个题目更进一步问你，**访问位现在分别是多少**——因为 5 号是刚好被装进来的，访问位是 1，所以现在这个具体时刻访问位也得知道。」（`[01:13:26]`–`[01:14:16]`）

⚠ 他念出的十六进制物理地址在转写里被拆碎（「A C F EFC」），无法还原成确切数值；本题只保留方法链（页号 5 → 页框 7 / 页框 2 → 拼偏移）。

4.2 第 19 题：二级页表 + 页目录（概念题，跨「进程/线程切换」）

「这个 19 题其实比较简单……**页目录号其实就是这个一级页号**；如果是二级页表，相当它是拆成了两部分：**有一级页号、二级页号，和偏移量**。当然一级页号有时候也叫做页目录号，然后二级页号有时候就叫做页号，注意一下这个名称。」（`[01:42:59]`–`[01:43:29]`）

```
第（1）问 —— 拼虚拟地址：从图里读出三段各占几位（本题 10 / 10 / 12 位），把题目给的三个片段
  分别写成 10 位、10 位、12 位二进制，拼起来再转十六进制。
  按讲稿念出的三个数字（6 / 6 / 8）拼出来是 0x01806008   ⚠ 见脚注
第（2）问 —— 页目录（一级页表）里存的是什么：「这个东西存放的应该是【物理地址】」（不是虚拟地址）
  · 进程切换：每个进程都有自己的页表 → 页目录的物理地址【会变】
  · 同一进程内的线程切换：线程共享进程资源，用的是同一张页表 → 【不会变】
第（3）问 —— 支持改进 CLOCK，页表项应增加哪些字段：【访问位】和【修改位】
```

他借这道题的图串了一遍跨科流程（`[01:45:44]`–`[01:49:10]`）：逻辑地址 → 先查**快表**，命中就直接拿页框号与偏移拼接；未命中再查**慢表（二级页表）**；两者目的都是拿到页框号 → 拼成物理地址 → 访问内存（内存的数据又有部分在 Cache 里）。图中物理地址中间定位到**两行**，故是**二路组相联**：「如果直接映射，中间应该是

行号，它只会定位到一行；那现在突然中间定位到两行，那应该是二路组相联」；比较器比较 Cache 标记与物理地址标记，**标记相等且有效位为 1 就命中**，再用**块内地址**取具体数据。他强调「**这个图一定要好好地多看一下**」。

⚠ 「6 / 6 / 8」三个数字来自转写，题面没有完整念出；拼出的 0x01806008 是按 10+10+12 位的定义算的，仅供参考，请以原题图为准。

4.3 第 51 题：改进CLOCK 的四类页面淘汰顺序

答案（背这个）

按 (A, M) 的取值，四类页面的淘汰先后次序：**(0,0) → (0,1) → (1,0) → (1,1)**。
直观理解：**先淘汰「又没访问过又没改过」的（连写回外存都不用，最省事）**，最后淘汰「又访问过又改过」的（既要写回、又刚用过）。

⚠ 他把顺序念成的那串数字在转写里是乱码（「0010011」「001 1001」），无法逐字复核；本顺序依据他 `[01:24:31]` 改口后的口述与 `[01:25:41]`–`[01:28:57]` 的四状态推导，并与教材一致。

4.4 第 52 题：一段 Peterson 风格的代码（选择题）

「看一下 52 题。这个题目的代码就是 Peterson 算法——**虽然没有明说，但是学过刚才的应该能看出来**：定义了两个标志，还有一个 turn 变量，初始的话都是 false。」
「首先它问你能不能实现互斥。**因为如果 flag 同时等于 true，它们会卡在 while 循环**……你（其实）想不出来一种情况说两个同时进到临界区，就是说两个 while 条件都不成立、同时进去，这样的（情况）是没有的，因为两个进程执行到这一步的时候，**flag0 和 flag1 就都等于 true 了**；然后 turn——**你到底谁是后执行的**：如果你先执行右边再执行左边，那最后 turn 就是 1，那 turn 就是 1 的话左边就进不去了，右边就能进到临界区；如果你是先执行左边再执行右边，那最后 turn 的值是 0，因为**后者会覆盖前者**，（所以）右边就进不去了，左边就会进去。**它只会让一个进到临界区。**」
「然后**因为有 turn 变量也不会有饥饿**；（如果）没有 turn 变量的话就会导致饥饿，两个都进不去；**有 turn 变量就能保证至少有一个进去。**」（`[02:29:05]`–`[02:33:34]`）

这段分析的两条通用套路（可直接迁移）

① **证不了「能互斥」就去找反例**：想出一种「两个 while 都不成立、同时进」的执行时序 = 不能互斥。
② **有「最后一刻被后者覆盖」的那个变量，就能打破僵局**：它能保证至少一个进程进得去（不会饥饿），代价是「谦让」——最后写的人吃亏。

5 统考真题精解：2021 年第 28 题与第 45 题

> **先说清楚年份这件事**
>
> **老师本讲全程没有说出任何一道题的统考年份**（讲稿里没有出现「××年真题」「统考」「真题」这类字眼）。他只用**讲义编号**指题：15、19、25、51、52、55。下面标注的年份，是整理时把老师讲的题面与公开的历年 408 真题**逐项比对**后认定的，依据逐条写在每题后面。**比对不牢靠的一律不标年份。**

5.1 2021 第 28 题 = 讲义第 55 题（请求分页 + 改进CLOCK）

题面：某请求分页存储系统页大小 4KB，按字节编址。给进程 P 分配 **2 个固定页框**，采用**改进型 Clock 置换算法**。页号 2 的存在位为 0（不在内存）；页 3 在页框 **60H**（存在位 1、访问位 1、修改位 0）；页 4 在页框 **80H**（存在位 1、访问位 1、修改位 1）。访问虚地址 **02A01H**，求物理地址。

> 「首先是请求分页，页面大小 4KB，按字节编址，4K 个地址的话说明……**这个偏移量得占 12 位。**」
> 「**存在位为一的就这两个，那说明（这个进程）刚好分给它的页框就是 60 和 80。**」
> 「你要访问的是 2，但是（刚才这两个页框）放的是 3 和 4……已经放满了，那就得置换。」
> 「那选谁置换呢，刚才分析过，**10 和 11，应该先选 10 置换出去**」→「所以最终……把 10 置换出去了」。
> 「如果访问位是 1，就把它改成 0，指针往后走；访问位是 1 就给它改成 0，指针往后走……现在的话访问位是 0 了，那就可以置换了。」（`[01:33:20]`–`[01:38:05]`）

```
① 页面 4KB → 偏移 12 位；虚地址 02A01H 的 bits19–12 = 02H = 页 2，低 12 位 = A01H
② 页 2 存在位为 0 → 缺页，要置换。两个候选：(A=1,M=0) 的页 3 与 (A=1,M=1) 的页 4
③ 改进 CLOCK：第一轮找 (0,0)——都不满足；第二轮找 (0,1) 并把扫过的 A 清 0——仍不满足；
   第三轮重复第一轮，此时页 3 的 (A,M) 已变成 (0,0) → 淘汰页 3
④ 页 2 装入页框 60H → 物理地址 = 60H × 4KB + A01H = 60A01H
```

> **他顺手讲的「第二次置换会怎样」（考场可能追问）**
>
> 「如果发生第二次置换，那你就应该（知道）因为刚才是把（那个页）置换了，**指针会指向下一个**，所以如果下次又要发生置换，其实（那几个）访问位现在就是 0 了，那就把这个位置换掉。当然如果你在刚才置换完之后，**中间又访问了一号**，那很有可能一号这个本来（访问位是 0，）你中间又访问了一下又把它变成 1 了，那再要置换的话可能流程就又不一样。」（`[01:14:16]`–`[01:15:13]`）

认定依据：公开的 2021 年 408 真题操作系统第 28 题（选择题）题干即「页大小为 4KB，按字节编址；给进程 P 分配 2 个固定的页框并采用改进型 Clock 置换算法」，标准答案为 **60A01H**，解析为「02H 页对应的页表项中存在位为 0……选择将 3 号页换出，将 2 号页放入 60H 页框」。这与老师讲的 **4KB 按字节编址 / 2 个固定页框 / 改进型 Clock / 存在位为 1 的是 60H 和 80H / 先淘汰 (1,0) 那一页 / 最终用 60 号页框** 六项全部吻合。⚠️ 老师本讲没有说年份，也没说这是真题。

5.2 2021 第 45 题 = 讲义第 25 题（整型信号量 + 开关中断）

题面：给出整型信号量 S 的 `wait()` / `signal()` 功能描述，以及用开 / 关中断指令实现信号量操作互斥的两种方法。问：(1) 为什么对 S 的访问必须互斥执行；(2) 方法 1 和方法 2 是否正确，不正确说明理由；(3) 用户程序能否使用开 / 关中断指令。

```
方法 1                          方法 2
wait(S) {                       wait(S) {
    关中断；                        关中断；
    while (S <= 0);                 while (S <= 0) {
    S = S - 1;                          开中断；
    开中断；                           关中断；
}                                   }
}                                   S = S - 1;
                                    开中断；
                                 }
两种方法的 signal(S) 相同：关中断； S = S + 1; 开中断；
```

(1)「（信号量 S 是一个全局变量，）**每一个进程都会对这个变量进行读操作或者写操作**……如果能让他们随便地去交叉执行，相当整个程序其实会乱掉。比方说这个 S 本来是 5，你第一（个进程）把它减一个 1，第二（个进程）也把它减一个 1，那很有可能他们第一（个）会在（原来的）基础上减一个 1，第二（个）也是在（原来的）基础上减一个 1，这样减完就变成 4 了；但实际上应该是第一（个）减完变成 4，第二（个）变成 3。**所以（这个题目你只要回答出）共享变量也是一个临界资源。**」

(2)「**方法一（错误）：你（先）把中断关了，如果这个变量小于 0，比方说 S 是 -2，它是小于零，那相当 P1 这个进程要一直执行这个循环；但是你现在又把中断关了，（CPU）切换不到别的进程上，所以这个 while 循环变成死循环了，退不出去。**」

「（方法二正确：）它在执行循环的时候又把中断打开了，一打开中断的时候，CPU 就能切换到（别的进程）……那你切换到（另一个进程），很有可能（它）去执行一个 signal 操作，把这个变量给它加个 1；**因为每次对这个变量的操作前后都要关中断**……这时候再切回来（就）发现这个（S）小于零就不成立了，P1 就能顺利去执行了。」（`[02:46:27]`–`[02:52:27]`）

(3)「开关中断指令是**权限很高的指令，是特权指令**……应该是在**内核态**（执行），用户程序的不行，应该回答出这个东西是特权指令。」（`[02:52:51]`–`[02:53:24]`，转写作「全线」= 权限）

这道题的三句标准答案（他给的）

(1) S 是被多个进程共享的**临界资源**（共享变量），多个进程都要对它读 / 写，若并发交叉执行会导致结果错误，故必须互斥。

(2) **方法 1 错误**：关中断后若 S≤0，while 会一直空转，而此时已关中断、不能切换进程 → **死循环**；**方法 2 正确**：循环体内先开中断（让别的进程能进来把 S 加 1）再关中断，能退出循环。

(3) **不能**：开 / 关中断是**特权指令**，只能在核心态（内核态）执行。

他还借这道题回补了整型信号量：「我们之前只讲了记录型信号量。如果看书的话你会发现这个整型信号量是这样的：**它是先写 while 循环，再让它减一**；之前（讲的）记录型信号量是**先把它减一，如果小于零再阻塞**。现在这个整型信号量是如果（S）已经小于等于零了，你让它卡在循环这一步就行——**所以卡在循环这一步，它并不是说 CPU 让出来的，而是一直去利用 CPU**，只不过用 CPU 干的是无意义的事情，反复执行这个循环。」（`[02:40:30]`–`[02:41:22]`）

认定依据：公开的 2021 年 408 真题操作系统第 45 题（解答题，7 分）题干即「给出整型信号量 S 的 wait() 和 signal() 操作的功能描述，以及采用开 / 关中断指令实现信号量操作互斥的两种方法」，三问与本题完全相同，标准答案要点为「信号量 S 是临界资源 / 方法 1 错误：关中断之后 while 语句会陷入死循环 / 方法 2 正确

/ 开中断和关中断指令都是特权指令」。尤其**方法 2 在 while 循环体内先开中断再关中断**这一形态与老师逐句讲的内容完全一致。⚠ 老师本讲没有说年份。

5.3 本讲结论与历年真题考点的对应（整理时比对，非老师口径）

年份	考点	对应本讲	认定依据 / 可信度
2021	请求分页 + 改进型 Clock 求物理地址（28 题）	§5.1（第 55 题）	**高**：题面六项要素逐项吻合，答案 60A01H 一致
2021	整型信号量 + 开关中断实现互斥（45 题）	§5.2（第 25 题）	**高**：三问与代码形态完全一致，标准答案要点逐条相同
2018	「可以实现让权等待的是」（Peterson / swap / 信号量 / TestAndSet，答案：信号量方法）	§3.10 表末行；他反复强调的结论	**中**：结论与老师口径一致，但⚠ **他没念这道题**，属整理时比对的推断性标注
2016	TSL / TS 指令伪代码：「等待进入临界区的进程不会主动放弃 CPU」	§3.10「TS 指令」一行	**中**：与他的分析同向，⚠ 他未提年份、未念选项
2010	Peterson 情形选择题：能保证互斥进入临界区、不会出现饥饿（只是不满足让权等待）	§4.4（第 52 题）	**低—中**：疑为同一题，但讲稿没念选项，**无法逐字比对**，仅作「可能对应」

2009 年以来 408 中「请求分页 / 页面置换」与「同步互斥」都是反复出现的考点区，但本资料**只标能逐项比对的年份**，不凭印象补年份。

6 易错点清单

#	易错点	正确做法 / 老师原话
1	把「缺页次数」当成「置换次数」	前几次缺页时还有空闲页框，**缺页不需要置换**；他说这是「缺页次数和置换次数的两个概念」
2	FIFO 的两个坑	① **FIFO 命中不改变队列顺序**（他本人先说错、后改口，见 §3.3）；② **FIFO 有 Belady 异常**——3 块 9 次、4 块 10 次，「物理块多了，你还缺的次数也变多了」
3	把缺页中断写成外中断 / 写在指令执行之后	缺页中断是**内中断（内部异常）**，产生在**指令执行期间**；一条指令可多次缺页

#	易错点	正确做法 / 老师原话
4	CLOCK 族的两处细节	① **只有发生淘汰指针才移动**，命中只把访问位改成 1；② 改进CLOCK 别忘修改位，淘汰顺序 **(0,0) → (0,1) → (1,0) → (1,1)**
5	新调入页面的访问位 / 修改位写错	新调入：**访问位 = 1、修改位 = 0**；中途被修改过才把修改位置 1
6	答题书写两个坑	① 置换后**不能**直接写「60 号页框里放的是 2 号页」——「**页表的编号一定是连续的**……你应该把 2 号这里改成 60，这时候同时有两个 60 了那怎么办，就应该把其中一个存在位置为 1，另一个存在位置为 0」；② 地址变换「考试千万不要写个答案……**要稍微写一下过程**」
7	把页框（帧）和计网的「帧」搞混	「这个帧和计算机网络的帧没有关系，但是都是一个字，一定要区分开」

#	易错点	正确做法 / 老师原话
8	概念混淆：驻留集 / 工作集、页表项字段	驻留集是**页框**的集合、工作集是**页面**的集合，**驻留集不能小于工作集**；请求分页页表项要**再加**状态位、访问字段、修改位、外存地址
9	把「双标志先检查 / 后检查」的判定搞反	先检查后置位（先检查）→ 可能**同时通过检查、同时进临界区** → 违背**忙则等待**；先置位后检查（后检查）→ 可能**两个都进不去** → 违背空闲让进 → **饥饿**
10	以为中断屏蔽在多处理机上也好用	「**（关中断）不是用于多处理系统**……在一个 CPU 上关中断，并不能防止其他 CPU 执行相同的临界区代码」
11	以为开关中断指令用户程序能用	它是**特权指令**，只能在内核态执行（ = 2021 年第 45 题第 3 问）
12	把 TS / Swap / 整型信号量当成能让权等待	都是 while 空转，**不能实现让权等待**；只有记录型信号量的 P 操作能
13	以为「能互斥」= 四条准则都满足	四条准则要分开判；如 Peterson 满足前三条、唯独不满足让权等待

7 考场速查卡

请求分页 · 必背 6 条

① 请求分页页表项 = 基本分页（页号 + 物理块号）**再加**：状态位、访问字段、修改位、外存地址。
② **缺页中断 = 内中断（内部异常），发生在指令执行期间，一条指令可多次缺页。**
③ 驻留集 = 页框的集合；工作集 = 页面的集合；**驻留集 ≥ 工作集**，否则频繁缺页。
④ 分配策略三种：固定分配局部置换 / 可变分配全局置换 / 可变分配局部置换。
⑤ 地址变换：**物理地址 = 页框号 × 页面大小 + 页内偏移**（ = 把页号换成页框号，偏移不动）。
⑥ 页面大小 4KB → 偏移 12 位；1KB → 10 位；页号位数 = 地址位数 − 偏移位数。

置换算法 · 30 秒决策

「淘汰谁」四问：**OPT 往后看**（最久才用到）/ **FIFO 看队列**（最先进来）/ **LRU 往前看**（最久没用）/ **CLOCK 转圈找访问位 0**（是 0 就换，是 1 就清 0 后移）。

改进CLOCK 四步：①找 (0,0)（不改标志位）→②找 (0,1) 并把扫过的 A 清 0 →③重复① →④重复②。**每次淘汰都从头走一遍。**

淘汰顺序：(0,0) → (0,1) → (1,0) → (1,1)。　**新调入：A=1, M=0。**

Belady 异常只在 FIFO。　**命中不动指针，只有淘汰才动指针。**　**缺页 ≠ 置换。**

互斥 · 一页纸结论

四条准则：空闲让进 / 忙则等待 / 有限等待 / **让权等待。**

四种软件方法：单标志法（能互斥，**必须交替**，违背空闲让进）| 双标志先检查（**不能互斥**，可能同时进，违背忙则等待）| 双标志后检查（能互斥，可能**都进不去 → 饥饿**，违背空闲让进、有限等待）| Peterson（**满足前三条，唯独不满足让权等待**）。

三种硬件方法：中断屏蔽（关中断；单 CPU 有效，**多处理机无效**）| TS 指令（原子操作，`old = *lock; *lock = TRUE; return old;`，**忙等**）| Swap 指令（原子操作，交换 `lock` 与局部 `key`，**忙等**）。二者的共同点：「**代码只是一些伪代码或者功能描述，他们是由硬件实现，不会被中断**」，本质没有区别。

让权等待只有一种实现：记录型信号量的 P 操作（S<0 时 block，进阻塞队列，让出 CPU）。整型信号量、单/双标志、Peterson、TS、Swap **全是忙等。**

答题规范（他反复说的）

大题写**过程**：置换算法写「谁最先进来 / 谁的访问位是几 / 所以淘汰谁」；地址变换写「页号几位、偏移几位、页框号怎么来的」。

概念题写**关键词**：共享变量 = 临界资源；缺页中断 = 内中断；开关中断 = 特权指令；页目录表项存的是物理地址。

附：本资料的来源与边界

项目	情况
来源	《408 各科大题知识点强化课》第 14 个分 P（课程内第 12 讲）「请求分页与软硬件互斥」，时长 **3 小时 12 分 25 秒**，全程本地 Whisper 转写。
时间戳覆盖	讲稿 `.md` 共 2286 行 / 约 12 万字节，时间戳从 `[00:00:04]` 到 `[03:12:03]`，即**全程**；其中 `[01:50:06]`–`[02:03:47]` 是**课间休息**（约 13 分 40 秒无内容）。
板书情况	**本讲没有可见板书**。所有表格、指针位置、页框状态、代码都是他口述的（转写里只有「先把东西我先画一下」「画在中间吧」这类话）。置换算法的表与 CLOCK 的状态演变，是按他逐步念出的操作**还原**的。
错字情况	错字极多且集中在术语上，已在文首列了两块「ASR 校正说明」。引用处均按上下文校正，并在括号内注明。
非考点段落	开头 `[00:00:04]`–`[00:08:10]` 是考研报名提醒、408 择校资料与院校地图、分数线公众号推荐；结尾 `[03:07:03]`–`[03:12:03]` 是复习节奏建议——**10 月 20 号左右开始做整套卷**，「前期的话我觉得（试卷）你可以三天一套，慢点做，做个五六套……后面就可以两天一套、一天一套」「前期一定要频率不要太高」；读者写者问题「**从来没考过**，属于大后期，四本书没复习完就先不用管」。**均与考点无关，仅在此备注。**
教材补充（明确不是老师口径）	① §3.9 的**缺页率与 EAT 公式**——本讲转写稿里**完全没有**；② §2.3 后段工作集的**标准定义**（「某段时间间隔内实际访问的页面的集合」，窗口概念），他说的是「经常访问的页面的集合」；③ OPT/LRU 表的后 14 列、FIFO 表的后 14 列与三种算法的**缺页总次数（9 / 15 / 12）**——他只推了前几步，其余是整理时按他给的规则算的；④ §3.7 对比表的「需要什么硬件位」一列，部分为教材归纳。
没有讲完 / 未覆盖	① 置换算法只讲了五种，**页面缓冲算法、请求分段、请求段页式**等未涉及；② 缺页处理的具体流程未展开；③ 互斥部分**没有讲管程**；④ 他明确说分页还有几道大题「没来得及讲」：「分页这边有几道题没来得及讲，到时候课下对着没讲的那些大题，我到时候会发一个资料」「所有我们上课没来得及讲的大题，我都会单独传一个视频讲解」（`[01:49:30]`、`[03:08:36]`）；⑤ 读者写者等未考过题型他明确让放到大后期。

⚠ **转写含糊、按上下文推断的地方（逐条列出）**

① **页面走向没有逐字念出。**OPT/FIFO/LRU 用的「7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1」是按他逐步念出的操作反推的：他说 `[00:47:11]`「7 0 1 2 7 0 1 2 这样」，随后每一步判断（2 换 7、3 换 1、0 命中、4 换 1…）与这一串完全自洽，⚠ **原串未在讲稿中完整出现。**

② **CLOCK 的页框顺序与指针起点。**他只在 `[00:48:32]` 说「最开始指针指向最前面」、`[00:51:18]` 说「指针的方向题干一定会告诉你（比如顺时针）」。本资料按「指针初始指向第一个页框（装 7 的那个）」还原，与他说「转了一圈，指针又回到原点了，这时候四个访问位全变成零」吻合。

③ **Belady 例子的页面走向。**他**只念了结论**（3 块 9 次、4 块 10 次），没有念这组例子的序列；本资料给出的「3 2 1 0 3 2 4 3 2 1 0 4」是教材上与 9 / 10 相符的经典例子，⚠ **序列本身不是他讲的。**

④ **第 15 题的十六进制答案。**他念物理地址时转写成了「A C F EFC」，`[01:13:23]` 处又念成「0B CLA」，都**无法还原成确切数值**；本资料只保留方法链（页号 5、FIFO 用页框 7、CLOCK 用页框 2）。

⑤ **第 19 题的三个数字「6 / 6 / 8」**来自 `[01:44:23]` 转写（「把这个 6 写成 10 位的二进制，把这个 6 也写成 10 位的二进制，把这个 8 写成 12 位的二进制」），题面没有完整念出；由此拼出的 0x01806008 仅供参考。

⑥ **第 55 题的题面要素**（页框 60H/80H、页 2/3/4、4KB）讲稿里念出了；虚地址 **02A01H** 与偏移 A01H 这两个具体数则是据 2021 年第 28 题补齐的——请以老师讲的逻辑为准，数字以原题为准。

⑦ **「开始」这个词。**他在 `[00:16:14]`、`[01:45:37]`、`[01:46:36]` 等处的「开始」在上下文里都与「快表」并列、且属第三章存储系统。**本资料一律按「Cache（高速缓冲存储器）」理解**，正文出现处都写了 Cache。⚠ **未得到讲稿直接确认。**

⑧ `[00:22:59]` **「逐流级主义是叫」**、`[00:24:44]` **「铸流级」**等一串同音写法，按上下文（「给一个进程分配的页框的集合」）统一确定为**驻留集。**

⑨ **「十中置换」= CLOCK 而非 LRU 的判定依据。**他在 `[01:10:58]` 说「如果采用的是十中置换算法，但目前这个题目十中置换算法考得比较简单」，紧接着 `[01:11:15]` 说「首先它给出了十中的初始（位置），其次告诉你（指针方向）是顺时针」——「初始指针位置 + 顺时针」正是 CLOCK 的题干要素（与 `[00:51:18]` 他讲 CLOCK 时的说法逐字对应），且该处前后文都在用访问位转圈，故判定为 CLOCK。⚠ **属推断。**

⑩ `[02:16:46]` **「那一开始 flag0 是错，是 force 没问题」**自相矛盾，与他自己 `[02:15:56]` 说的「FLAG0 和 FLAG1 都是 FALSE」冲突；本资料按初值 false 处理，不采信该句。

⑪ **第 52 题的题面代码没有念出。**他只描述「定义了两个标志，还有一个 turn 变量，初始都是 false」，本资料据此还原成标准 Peterson 结构，**只在引用他的分析时给出结构描述，未擅自写代码。**该题是否就是 2010 年那道 Peterson 真题，**无法确证**（讲稿没念选项）。

⑫ **互斥判定表（§3.10）**的 ✓/✗ 只覆盖他明确口述过的准则组合。

整合自 3 小时 12 分 25 秒网课讲稿（本地 Whisper 转写，全程无板书，代码与表格按口述还原）· 未上传任何材料到第三方服务 · 生成于 2026-10-10